CAPE PENINSULA UNIVERSITY OF TECHNOLOGY

SOFTWARE DESIGN 2 | 2026

AutoVault Data Backup Scheduler

FINAL PROJECT REPORT

Group Member	Assigned Role
Aphiwe Mtshali	Project Lead and Integration
Khano Mavhandu	Core Backup Developer
Luyema Maqasha	Scheduling and Retention Developer
Orinea Nemahate	Change Detection and Algorithms Developer
Ampfarisaho Thukhutha	Model, Storage and UI Developer
Kelebogile Moswane	QA, Logging and Documentation Lead

ABSTRACT

AutoVault is a Java-based data backup scheduling application developed to help users protect important files against accidental deletion, unwanted modifications and data loss.

The system allows users to select folders, specify backup destinations and configure automatic backup intervals. It maintains multiple backup versions so that users can restore an earlier copy of their files when necessary.

AutoVault provides a graphical user interface developed using Java Swing. Its implementation includes folder scanning, change detection, backup creation, scheduling, version retention, restoration, persistent job storage and activity logging.

The project applies object-oriented programming principles, data structures, searching and sorting algorithms, file handling, exception handling and multithreading.

Testing covered normal operations and invalid inputs. The automated test results recorded **25 successful tests and zero failures**, demonstrating that the tested backup, restoration, version management, persistence and error-handling operations behaved as expected.

The completed application provides a practical implementation of the core requirements of a scheduled backup and recovery system.

1. INTRODUCTION

1.1 Background

Data loss is a common problem for individuals and organisations that store important information electronically.

Files may be lost because of accidental deletion, hardware problems, software errors or unwanted modifications.

Manual backups can reduce these risks, but they rely on users remembering to create copies regularly. Users may also overwrite previous backups, making it difficult to recover an earlier version of a file.

An automated backup application helps address these problems by performing backups at scheduled intervals and maintaining a history of saved versions.

1.2 Problem Statement

Traditional manual backup practices present several challenges:

- Users may forget to back up important information.
- Repeated copying requires time and effort.
- Existing backup files may be overwritten.
- Recovering an earlier file version may be difficult.
- Users may not know whether a backup operation succeeded or failed.

These challenges create a need for a system that automates backup operations while preserving multiple versions for recovery.

1.3 Project Aim

The aim of AutoVault is to develop a Java application that enables users to schedule folder backups, maintain multiple backup versions and restore previously saved information.

1.4 Project Objectives

The objectives are to:

1. Develop an interface that allows users to select folders for backup.
2. Allow users to configure backup destinations and intervals.
3. Implement automatic backup scheduling.
4. Detect changes before creating another backup version.
5. Maintain multiple backup versions.
6. Allow users to restore selected versions.
7. Record backup operations and errors.
8. Save backup configurations between application sessions.
9. Apply appropriate algorithms and data structures.
10. Test normal operations and error conditions.

2. PROJECT REQUIREMENTS

The original project specification states:

Data Backup Scheduler: Build a system that allows users to select folders to back up automatically at set intervals. It should keep multiple backup versions for recovery.

The AutoVault implementation addresses these requirements through the following features.

Requirement	AutoVault Implementation
Folder selection	Users select source and destination folders through the interface.
Scheduled backups	The scheduling component runs jobs according to configured intervals.
Multiple versions	Successful changed backups create separate backup versions.
Recovery	Users can select a previous version and restore its contents.
Change detection	Unchanged folders are skipped to avoid unnecessary duplicate versions.
Version retention	The system keeps the five most recent versions of each job.

Requirement	AutoVault Implementation
Persistent configuration	Job information and version history are saved and loaded.
Logging	Backup events, warnings and failures are recorded.

The first four requirements form the essential functionality described in the assignment. The additional features support reliability, usability and maintainability.

3. PROJECT TEAM AND RESPONSIBILITIES

The project was divided into six roles to support collaboration and integration.

Team Member	Role	Main Responsibilities
Aphiwe Mtshali	Project Lead and Integration	Team coordination, repository management, BackupManager, Main and integration of team contributions.
Khano Mavhandu	Core Backup Developer	FileScanner, BackupEngine and RestoreService.
Luyema Maqasha	Scheduling and Retention Developer	BackupScheduler, ScheduledTask and RetentionPolicy.
Orinea Nemahate	Change Detection and Algorithms Developer	ChangeDetector, TimestampDetector, HashDetector and VersionSorter.
Ampfarisaho Thukhutha	Model, Storage and UI Developer	BackupJob, BackupVersion, job persistence, MainWindow and CliMenu.

Team Member	Role	Main Responsibilities
Kelebogile Moswane	QA, Logging and Documentation Lead	BackupLogger, testing, UML maintenance, documentation and presentation preparation.

The team combined its components into a single NetBeans application. Integration required the classes to use consistent package names, method signatures and shared data models.

4. SYSTEM DESIGN

4.1 Design Approach

AutoVault follows an object-oriented design in which different classes handle distinct responsibilities.

The application separates its main functions into four packages:

- core
- model
- ui
- util

This separation makes the system easier to understand, test and maintain.

4.2 Package Structure

Core Package

The core package contains the main backup functionality.

Its classes and interface are:

1. BackupEngine
2. BackupManager
3. BackupScheduler
4. ChangeDetector
5. FileScanner
6. HashDetector
7. RestoreService
8. RetentionPolicy
9. TimestampDetector
10. VersionSorter

The package controls backup execution, scheduling, restoration, version sorting and retention.

Model Package

The model package contains three classes:

1. BackupJob
2. BackupVersion
3. ScheduledTask

These classes represent backup configurations, stored versions and scheduled operations.

UI Package

The ui package contains:

1. Main
2. MainWindow
3. CliMenu
4. AutoVaultDemo
5. AutoVaultRole6Prototype

These classes provide the application entry point, graphical interface, console interface, demonstration functionality, and QA and logging window.

Util Package

The util package contains:

1. BackupLogger
2. LogEntry

These classes manage application event records.

4.3 UML Class Diagram

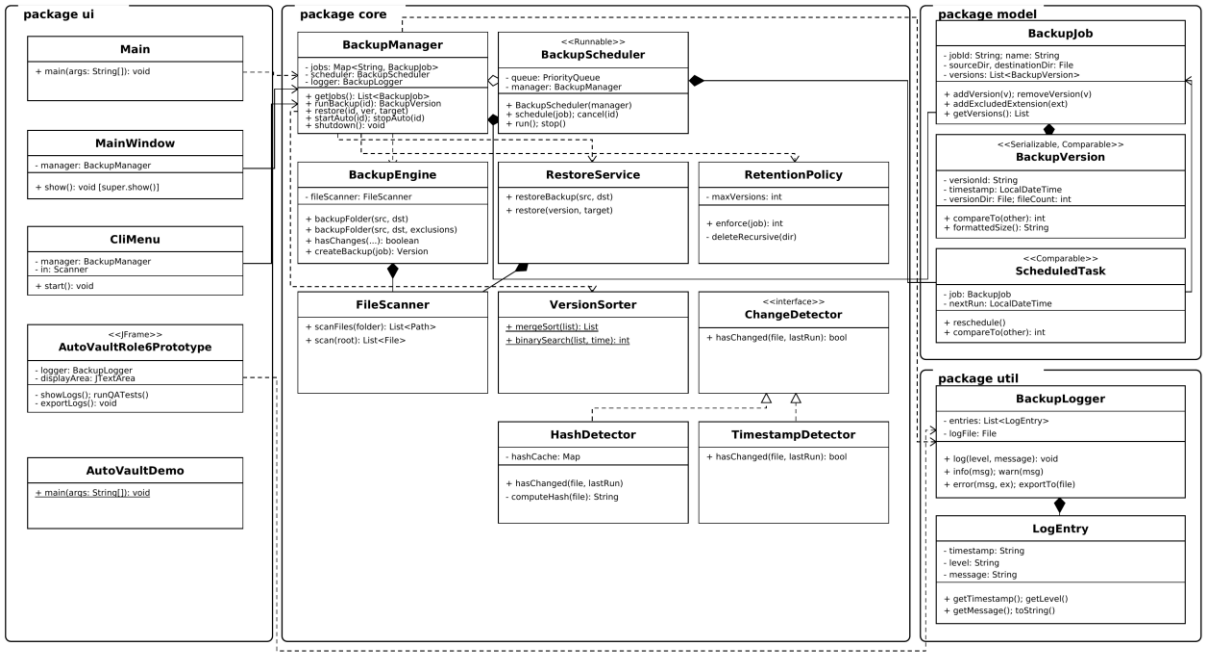


Figure 1: AutoVault UML class diagram.

The UML class diagram illustrates the classes, attributes, methods and relationships within the application.

The main relationships include:

- MainWindow communicates with BackupManager.
- BackupManager manages multiple BackupJob objects.
- Each BackupJob maintains a collection of BackupVersion objects.
- BackupManager uses BackupEngine to create backups.
- BackupScheduler manages ScheduledTask objects.
- Each ScheduledTask refers to a BackupJob.
- RestoreService restores the contents of selected backup versions.
- BackupLogger stores LogEntry records.
- HashDetector and TimestampDetector implement the ChangeDetector interface.

4.4 Object-Oriented Programming Principles

Encapsulation

Classes store their internal data in fields and provide methods for controlled access.

For example, BackupJob manages its configuration and version list through methods.

Abstraction

The ChangeDetector interface defines a common change-detection operation.

Its implementations provide different approaches to detecting file changes.

Polymorphism

HashDetector and TimestampDetector implement the same interface, allowing alternative change-detection implementations to share a common contract.

Modularity

Backup processing, scheduling, restoration and logging are implemented in separate components.

This reduces the number of responsibilities assigned to any individual class.

5. SYSTEM IMPLEMENTATION

5.1 Development Environment

AutoVault was developed using:

Component	Technology
Programming language	Java
Development environment	Apache NetBeans
User interface	Java Swing
File operations	Java file I/O and NIO APIs
Job persistence	Java object serialization
Activity logging	Text-based log file
Testing	Java automated test runner and application-level QA checks

5.2 Main Application Interface

The main user interface is implemented by MainWindow.

It provides controls for:

- Adding a backup job.
- Running a backup immediately.
- Viewing backup versions.
- Restoring a selected version.
- Removing backup jobs.
- Saving configuration changes.
- Accessing QA and logging functionality.

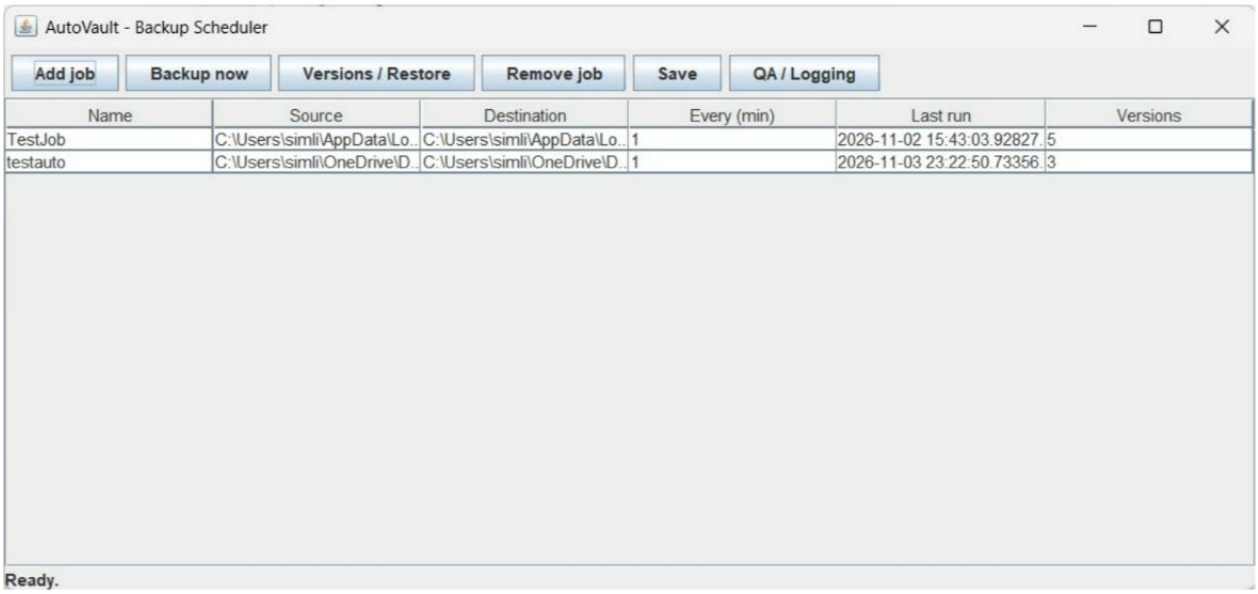


Figure 2: AutoVault backup management interface.

5.3 Creating a Backup Job

A backup job contains information such as:

- Job identification number.
- Job name.
- Source folder.
- Destination folder.
- Backup interval.
- Enabled status.
- Last execution time.
- List of stored backup versions.

The BackupJob class manages this information.

Input validation prevents invalid configurations, including empty job names, intervals below one minute, identical source and destination folders, and destinations located inside their own source folders.

5.4 Backup Processing

BackupManager coordinates the backup process.

When a backup is requested, the manager:

1. Finds the selected backup job.
2. Checks whether its source folder exists.
3. Uses BackupEngine to determine whether the contents have changed.
4. Skips the operation when no changes are detected.
5. Creates a new version when changes are present.
6. Adds that version to the job's history.
7. Applies the retention policy.

8. Saves the updated job information.
9. Records the outcome in the application log.

BackupEngine uses Java file-handling operations to create directories and copy files.

The implementation can also preserve empty directories and exclude selected file extensions.

5.5 Change Detection

Change detection prevents the application from storing another identical snapshot when the source folder has not changed.

The current BackupEngine compares source contents with the latest saved version. Its comparison logic includes Java's Files.mismatch() operation.

The project also includes separate TimestampDetector and HashDetector classes, which implement the ChangeDetector interface.

These demonstrate alternative approaches to detecting file modifications.

5.6 Automatic Backup Scheduling

The BackupScheduler class implements Java's Runnable interface.

It maintains scheduled operations in a priority queue.

Each ScheduledTask stores:

- A reference to its backup job.
- The next scheduled execution time.

The scheduler checks for due tasks, runs the relevant backup operation and reschedules active jobs.

The configured interval determines when the next execution becomes due.

Because scheduling runs within the application process, AutoVault must remain running for automatic backups to continue.

5.7 Backup Version Management

Every successful backup that detects changes creates a new backup version.

The BackupVersion class stores information about the saved snapshot, including:

- Version identifier.
- Creation timestamp.
- Backup directory.
- File count.
- Total size.

BackupManager applies a retention policy that keeps the **five most recent versions**.

Older versions are removed when the number of saved versions exceeds this limit.

This prevents version history from growing without restriction, although users must understand that older snapshots will eventually be deleted.

5.8 File Restoration

The RestoreService class handles recovery.

Users can select an existing backup version and choose a destination folder.

The service then copies the contents of the selected version to that location.

This allows recovery of an older file state without requiring the user to manually locate and copy the saved files.

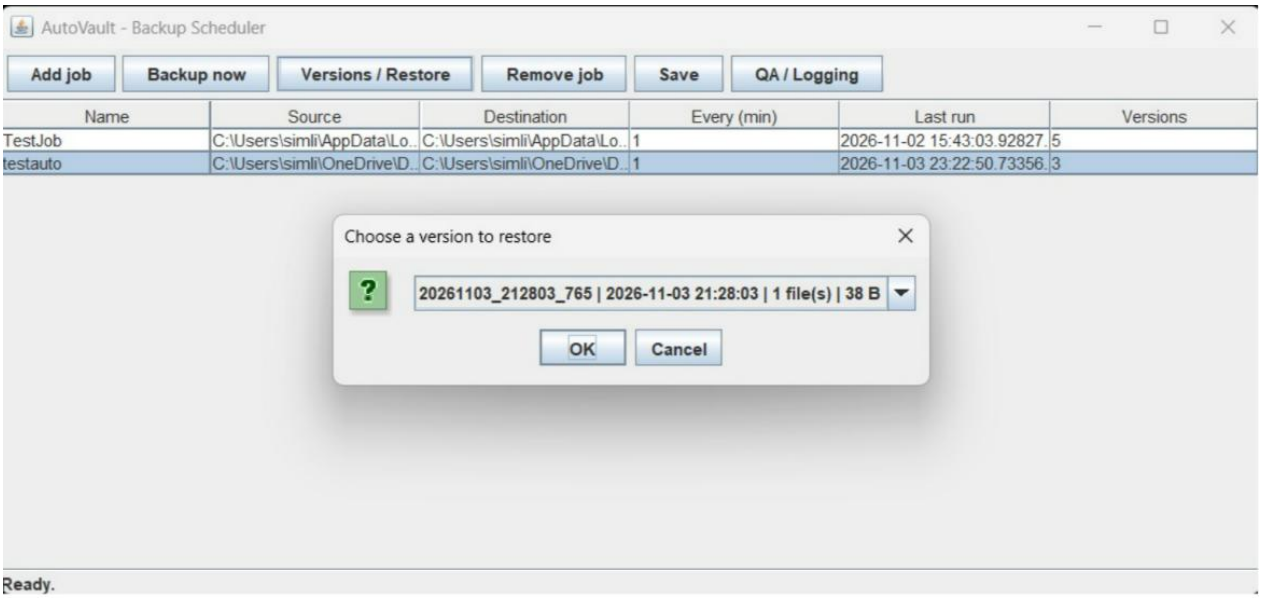


Figure 3: Selecting a backup version for restoration.

5.9 Saving and Loading Jobs

AutoVault saves backup job information using Java object serialization.

The application stores job data in a file named jobs.dat.

When the application starts, it can load previously saved jobs and their version information.

Enabled jobs are scheduled again during normal application startup.

This allows users to continue working with previously configured backups after restarting AutoVault.

5.10 Activity Logging

The BackupLogger class records events in AutoVault.log.

The logger supports:

- Information messages.
- Warning messages.
- Error messages.
- Retrieving recorded entries.
- Exporting logs.

Examples of events include successful backups, skipped backups, restoration results, job changes and unexpected failures.

The QA and Logging window allows users to inspect recorded events.

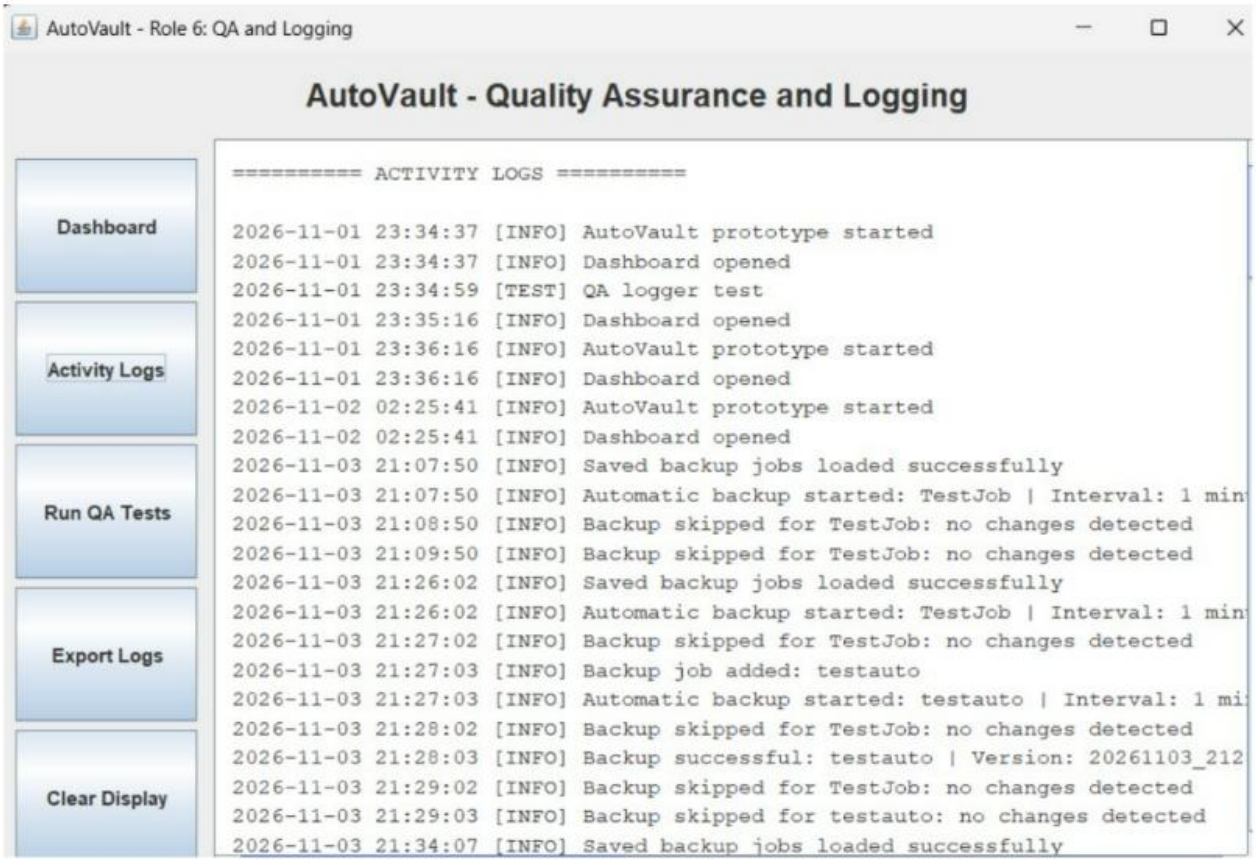


Figure 4: AutoVault quality assurance and logging interface.

6. ALGORITHMS AND DATA STRUCTURES

AutoVault applies several programming concepts covered in Software Design 2.

6.1 HashMap

BackupManager maintains backup jobs using a map associated with each job's unique identifier.

The implementation uses HashMap.

This allows the application to retrieve a job using its ID without searching through the complete job list manually.

6.2 Priority Queue

BackupScheduler uses PriorityQueue<ScheduledTask>.

The scheduled tasks are ordered according to their next execution time.

This supports efficient selection of the next backup task that becomes due.

6.3 Merge Sort

VersionSorter implements merge sort for ordering backup versions chronologically.

The implementation divides the version list into smaller sections, sorts them and combines the results.

The manager uses this functionality when listing versions.

Merge sort has a time complexity of:

O(nlog n)

6.4 Binary Search

VersionSorter also includes a binary-search method for finding a timestamp within a sorted list of backup versions.

Binary search repeatedly narrows the search interval.

Its time complexity is:

O(log n)

The current application's restoration operation identifies a selected version by its version ID using a list traversal. Therefore, the binary-search method is an implemented algorithm utility rather than the main restoration lookup mechanism.

6.5 File Handling

The application uses Java file operations to:

- Traverse folders.
- Create directories.
- Copy files.
- Compare file contents.
- Restore previously backed-up files.
- Save application configuration.
- Write log records.

6.6 Exception Handling

Exception handling allows the application to report failures without unexpectedly terminating normal execution.

For example, AutoVault handles missing source folders and invalid restoration requests by reporting errors and recording relevant log messages.

7. TESTING AND VALIDATION

7.1 Testing Objectives

Testing was performed to determine whether the implemented features behaved correctly under normal conditions and invalid inputs.

The main objectives were to verify:

- Backup job creation.
- Version creation.
- Change detection.
- Restoration.
- Version retention.
- Persistent storage.
- Logging.

- Input validation.
- Safe handling of errors.

7.2 Testing Approach

The project includes AutoVaultTests.java, an automated test runner.

Tests were divided into normal-operation tests and edge-input tests.

The saved test results contain 25 completed checks.

7.3 Normal-Operation Tests

Test	Expected Result	Result
N1	A valid backup job can be added.	Pass
N2	The first backup creates a version.	Pass
N3	Editing a file allows a second version to be created.	Pass
N4	Backup versions are listed chronologically.	Pass
N5	An older version can restore its original contents.	Pass
N6	Retention keeps the five newest versions after eight backups.	Pass
N7	Removed versions are deleted from disk.	Pass
N8	Jobs and version history survive saving and reloading.	Pass
N9	Successful backup activity is recorded in the log.	Pass
N10	Files with spaces and brackets in their names can be backed up and restored.	Pass

7.4 Edge-Input Tests

The remaining 15 tests checked invalid and unusual situations, including:

- Empty job names.
- Missing folder selections.
- Zero and negative intervals.
- Identical source and destination folders.
- Backup destinations located inside source folders.

- Unknown backup jobs.
- Unknown backup versions.
- Deleted source folders.
- Empty source folders.
- Repeated backups without changes.
- Restoration into a destination that does not yet exist.
- Attempts to back up a job after removal.
- Logging of a failed backup operation.

All 15 edge-input checks passed.

7.5 Test Results

Category	Passed	Failed
Normal-operation tests	10	0
Edge-input tests	15	0
Total	25	0

"Recorded 25 successful automated tests, alongside 15 manual and QA logging tests

The test runner reported:

Passed: 25

Failed: 0

This represents a 100% pass rate for the 25 tests executed.

The result provides evidence that the tested operations met their expected outcomes.

```
Source folder does not exist: C:\Users\simli\AppData\Local\Temp\autovault-tests-1793918393939\vanishing\source
PASS  E10  Backup when the source folder was deleted fails safely
        observed: result: null
PASS  E15  That failed backup is recorded in AutoVault.log
        observed: AutoVault.log size: 17664 bytes before, 17839 bytes after
Backup completed.
PASS  E11  Backup of an empty folder does not crash
        observed: result: version created
Backed up: notes.txt
Backed up: report.txt
Backup completed.
No changes since the last backup.
PASS  E12  A second backup with no changes is skipped
        observed: 1 version(s) after two backups with no edit in between
Restored folder:
Restored file: notes.txt
Restored file: report.txt
Restore completed successfully.
Restore finished.
PASS  E13  Restoring into a folder that does not exist yet works
        observed: restored file found: true
Job not found: 66b6311d-55ae-4109-a822-714b316cc8c1
PASS  E14  A removed job cannot be backed up
        observed: jobs left: 3

=== SUMMARY ===
Passed: 25
Failed: 0
Results saved to C:\Users\simli\OneDrive\Documentos\NetBeansProjects\AutoV\test-results.txt
BUILD SUCCESSFUL (total time: 11 seconds)
```

Figure 5: AutoVault automated testing summary.

8. CHALLENGES AND SOLUTIONS

8.1 Development Environment Compatibility

During development, an older NetBeans and Apache Ant configuration produced a build error relating to Java version compatibility.

The development environment was updated, after which compilation and testing could continue.

This highlighted the importance of matching the Java development kit, IDE and build tools.

8.2 Application Integration

Different team members developed components responsible for separate areas of the application.

Integration required consistent package declarations, imports, class names and method calls.

The team organised production classes into the core, model, ui and util packages.

8.3 QA Window Behaviour

An earlier version of the QA window used a close operation that could terminate the entire application when the QA window was closed.

The window behaviour was changed so that closing the QA interface disposes of that window without closing the main AutoVault application.

8.4 Handling Invalid Backup Configurations

Users may select invalid paths or enter unsupported intervals.

Validation was introduced to reject these configurations before they result in backup operations.

8.5 Preventing Unnecessary Backups

Creating another full copy when no source files have changed wastes storage.

The backup engine therefore checks for changes and skips unnecessary version creation.

9. LIMITATIONS AND FUTURE IMPROVEMENTS

Although the application meets the main project requirements, several improvements could be considered.

9.1 Background Scheduling

Automatic scheduling currently runs within the application process.

A future version could provide an operating-system service that continues scheduling without requiring the graphical application to remain open.

9.2 Backup Storage Options

Future versions could support external drives, network storage or cloud destinations.

9.3 Backup Security

Additional security features could include encrypted backup storage and stronger access controls.

9.4 More Extensive Testing

Further testing could investigate very large folders, long-running scheduled jobs, access-permission errors, files being modified during backup and recovery after unexpected shutdowns.

9.5 Storage Efficiency

AutoVault currently preserves full backup versions when changes are detected.

Incremental or differential storage could reduce the amount of duplicated data.

10. PROJECT ACHIEVEMENTS

The AutoVault project achieved the following outcomes:

1. Developed a Java based backup scheduling application.
2. Implemented a functional graphical user interface.
3. Allowed users to create and manage backup jobs.
4. Added interval-based automatic scheduling.
5. Implemented backup version creation and history management.
6. Supported restoration of previous versions.
7. Added change detection to skip unnecessary backups.
8. Implemented a five version retention policy.
9. Added persistent storage of backup jobs.
10. Integrated activity logging and a QA interface.
11. Applied relevant data structures and algorithms.
12. Recorded 25 successful automated tests.

The project provided practical experience in object-oriented design, software integration, file handling, algorithm implementation and software testing.

11. CONCLUSION

AutoVault was developed to address the need for scheduled data backups and recovery of earlier file versions.

The application allows users to configure backup jobs, run backups manually or automatically, maintain multiple versions and restore previously saved data.

Its implementation demonstrates several important Software Design 2 concepts, including object-oriented programming, interfaces, collections, scheduling, algorithms, file operations, exception handling and persistent storage.

The completed automated tests reported 25 passes and zero failures across normal operations and edge-input scenarios.

The project also strengthened the group's understanding of collaborative software development, integration, debugging and documentation.

Overall, AutoVault satisfies the central requirements of the assigned Data Backup Scheduler project and provides a foundation for further improvements.

REFERENCES

Oracle (2025a) Files (Java SE 24 and JDK 24). Available at: https://docs.oracle.com/en/java/javase/24/docs/api/java.base/java/nio/file/Files.html (Accessed: 1 November 2026).

Oracle (2025b) Map (Java SE 24 and JDK 24). Available at: https://docs.oracle.com/en/java/javase/24/docs/api/java.base/java/util/Map.html (Accessed: 1 November 2026).

Oracle (2025c) ObjectOutputStream (Java SE 24 and JDK 24). Available at: https://docs.oracle.com/en/java/javase/24/docs/api/java.base/java/io/ObjectOutputStream.html (Accessed: 4 November 2026).

Oracle (2025d) PriorityQueue (Java SE 24 and JDK 24). Available at: https://docs.oracle.com/en/java/javase/24/docs/api/java.base/java/util/PriorityQueue.html (Accessed: 5 November 2026).

GITHUB LINK

Aphiwe06-tech/AutoVault